JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent.atomic

Class AtomicStampedReference<V>

java.lang.Object
  java.util.concurrent.atomic.AtomicStampedReference<V>

Type Parameters:

`V - The type of object referred to by this reference`

```
public class AtomicStampedReference<V>
extends Object
```

An `AtomicStampedReference` maintains an object reference along with an integer "stamp", that can be updated atomically.

Implementation note: This implementation maintains stamped references by creating internal objects representing "boxed" [reference, integer] pairs.

Since:

`1.5`

Constructor Summary

Constructors

Constructor	Description
`AtomicStampedReference(V initialRef, int initialStamp)`	Creates a new `AtomicStampedReference` with the given initial values.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
boolean	**attemptStamp**(**V** expectedReference, int newStamp)	Atomically sets the value of the stamp to the given update value if the current reference is == to the expected reference.
boolean	**compareAndSet**(**V** expectedReference, **V** newReference, int expectedStamp, int newStamp)	Atomically sets the value of both the reference and stamp to the given update values if the current reference is == to the expected reference and the current stamp is equal to the expected stamp.
V	**get**(int[] stampHolder)	Returns the current values of both the reference and the stamp.
V	**getReference**()	Returns the current value of the reference.
int	**getStamp**()	Returns the current value of the stamp.
void	**set**(**V** newReference, int newStamp)	Unconditionally sets the value

		of both the reference and stamp.
boolean	**weakCompareAndSet**(V expectedReference, V newReference, int expectedStamp, int newStamp)	Atomically sets the value of both the reference and stamp to the given update values if the current reference is == to the expected reference and the current stamp is equal to the expected stamp.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

AtomicStampedReference

```
public AtomicStampedReference(V initialRef,
                              int initialStamp)
```

Creates a new AtomicStampedReference with the given initial values.

Parameters:

initialRef - the initial reference

initialStamp - the initial stamp

Method Detail

getReference

```
public V getReference()
```

Returns the current value of the reference.

Returns:

`the current value of the reference`

getStamp

```
public int getStamp()
```

Returns the current value of the stamp.

Returns:

`the current value of the stamp`

get

```
public V get(int[] stampHolder)
```

Returns the current values of both the reference and the stamp. Typical usage is `int[1] holder; ref = v.get(holder); .`

Parameters:

`stampHolder - an array of size of at least one. On return, stampholder[0] will hold the value of the stamp.`

Returns:

`the current value of the reference`

weakCompareAndSet

```
public boolean weakCompareAndSet(V expectedReference,
                                 V newReference,
                                 int expectedStamp,
                                 int newStamp)
```

Atomically sets the value of both the reference and stamp to the given update values if the current reference is == to the expected reference and the current stamp is equal to the expected stamp.

May fail spuriously and does not provide ordering guarantees, so is only rarely an appropriate alternative to `compareAndSet`.

Parameters:

`expectedReference - the expected value of the reference`

`newReference - the new value for the reference`

`expectedStamp - the expected value of the stamp`

`newStamp - the new value for the stamp`

Returns:

`true if successful`

compareAndSet

```
public boolean compareAndSet(V expectedReference,
                             V newReference,
                             int expectedStamp,
                             int newStamp)
```

Atomically sets the value of both the reference and stamp to the given update values if the current reference is == to the expected reference and the current stamp is equal to the expected stamp.

Parameters:

`expectedReference - the expected value of the reference`

`newReference - the new value for the reference`

`expectedStamp - the expected value of the stamp`

`newStamp - the new value for the stamp`

Returns:

`true if successful`

set

```
public void set(V newReference,
                int newStamp)
```

Unconditionally sets the value of both the reference and stamp.

Parameters:

`newReference - the new value for the reference`

`newStamp - the new value for the stamp`

attemptStamp

```
public boolean attemptStamp(V expectedReference,
                            int newStamp)
```

Atomically sets the value of the stamp to the given update value if the current reference is == to the expected reference. Any given invocation of this operation may fail (return `false`) spuriously, but repeated invocation when the current value holds the expected value and no other thread is also attempting to set the value will eventually succeed.

Parameters:

`expectedReference - the expected value of the reference`

`newStamp - the new value for the stamp`

Returns:

`true if successful`

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.